MDM Management Assistant

MacOS MDM Enrollment Detection & Authorized Remediation Workflow

Portfolio Project Report
Service Desk / IT Support / Endpoint Management

Project scope: Authorized diagnostic and reporting only. The project does not bypass or defeat MDM security controls.

1. Project Overview

This project demonstrates a practical Service Desk workflow for investigating Mobile Device Management (MDM) enrollment on macOS. It uses built-in macOS commands through Python and Bash to collect management information, create a diagnostic report, and document the administrator-led remediation process.

2. Objectives

• Detect whether a Mac is enrolled in MDM.

• Collect macOS and configuration-profile information.

• Generate a structured diagnostic report.

• Document an authorized unenrollment/release workflow.

• Verify the device status after administrator remediation.

3. Technology Stack

Technology	Use
Python 3	Automation and JSON reporting
Bash	macOS diagnostic commands
macOS profiles utility	Enrollment/profile inspection
JSON	Structured diagnostic output
Git/GitHub	Version control and portfolio presentation

4. Architecture / Workflow

User → Python/Bash diagnostic layer → macOS management commands → collected results → JSON report → authorized administrator review → release/unenrollment by the organization → verification scan.

5. Complete Python Source Code

```
#!/usr/bin/env python3
"""
MDM Management Assistant
Purpose: authorized macOS MDM diagnostics and reporting.
This tool DOES NOT bypass or defeat MDM controls.
"""

import json
import platform
import subprocess
from datetime import datetime
from pathlib import Path

REPORT_DIR = Path("reports")
REPORT_DIR.mkdir(exist_ok=True)

def run(cmd):
    try:
        p = subprocess.run(cmd, capture_output=True, text=True, timeout=15)
        return {"returncode": p.returncode, "stdout": p.stdout.strip(), "stderr": p.stderr.strip()}
    except Exception as e:
        return {"returncode": -1, "stdout": "", "stderr": str(e)}

def main():
    if platform.system() != "Darwin":
        print("This diagnostic utility is designed for macOS.")
        return

    checks = {
        "system_version": run(["sw_vers"]),
        "profiles_status": run(["profiles", "status", "-type", "enrollment"]),
        "profiles_list": run(["profiles", "list"]),
    }

    report = {
        "generated_at": datetime.now().isoformat(timespec="seconds"),
        "hostname": platform.node(),
        "platform": platform.platform(),
        "checks": checks,
        "note": "Use only on devices you own or are authorized to administer."
    }

    output = REPORT_DIR / "mdm_diagnostic_report.json"
    output.write_text(json.dumps(report, indent=2), encoding="utf-8")

    print("MDM diagnostic completed.")
    print(f"Report saved to: {output}")
    print("\nEnrollment check:")
    print(checks["profiles_status"]["stdout"] or checks["profiles_status"]["stderr"])

if __name__ == "__main__":
    main()
```

6. Complete Bash Source Code

```
#!/bin/bash
# Authorized MDM information collection helper.
# Does not bypass, remove, or defeat MDM controls.

set -u

echo "=== MDM Management Assistant ==="
echo "Date: $(date)"
echo

echo "=== macOS Version ==="
sw_vers
echo

echo "=== MDM Enrollment Status ==="
profiles status -type enrollment
echo

echo "=== Installed Configuration Profiles ==="
profiles list
echo

echo "=== Next step ==="
echo "If the device is organization-managed, contact the authorized MDM administrator"
echo "to release/unenroll the device. Do not attempt to bypass management controls."
```

7. Installation & Usage

````
# MDM Management Assistant

A Service Desk / IT Support portfolio project for authorized macOS MDM diagnostics.

## Features
- Detects macOS platform information.
- Checks MDM enrollment status.
- Lists configuration profiles visible to the authorized user.
- Produces a JSON diagnostic report.
- Provides an administrator-led remediation workflow.

## Safety and scope
This project is intentionally designed for authorized administration. It does not bypass,
crack, defeat, or remove MDM restrictions without authorization.

## Requirements
- macOS
- Python 3
- Terminal access
- Appropriate permission to inspect the device

## Run
```bash
python3 mdm_checker.py
```

Or:
```bash
chmod +x collect_mdm_info.sh
./collect_mdm_info.sh
```

The Python tool writes:
`reports/mdm_diagnostic_report.json`

## Authorized remediation workflow
1. Confirm ownership/authorization.
2. Record the device serial number and enrollment status.
3. Identify the organization's MDM administrator.
4. Ask the administrator to release/unenroll the device.
5. Re-run the diagnostic tool.
6. Record the post-remediation status.

## Skills demonstrated
Python, Bash, macOS administration, MDM concepts, endpoint troubleshooting,
report generation, Service Desk documentation.
````

8. Sample Output


```
{
  "generated_at": "2026-10-05T22:00:00",
  "hostname": "sample-mac",
  "platform": "macOS sample environment",
  "checks": {
    "profiles_status": {
      "stdout": "Sample output: Enrollment status should be reviewed by an authorized administrator."
    }
  },
  "note": "Sample report for portfolio demonstration; not a real device report."
}
```


The sample output is intentionally labeled as demonstration data and should not be presented as a real device scan.

9. Authorized Remediation Procedure

• Confirm the device is yours or that you have written authorization to administer it.

• Collect the enrollment status and relevant device details.

• Contact the organization's MDM administrator or IT asset-management team.

• Request official release/unenrollment from the organization's MDM system.

• After authorized removal, run the diagnostic again and retain the before/after reports.

• Document the resolution in the Service Desk ticket.

10. Resume Description

MDM Management Assistant — Python, Bash, macOS
Developed an authorized macOS MDM diagnostic utility that checks enrollment status, collects configuration-profile information, generates structured JSON reports, and documents an administrator-led remediation and verification workflow.

11. Interview Explanation

“I built this project to understand endpoint management from a Service Desk perspective. The utility automates initial MDM diagnostics using Python and Bash, creates a report for troubleshooting, and keeps the actual unenrollment under the control of the authorized MDM administrator. This taught me how to document incidents, collect evidence, follow security boundaries, and verify remediation.”

12. Important Security Boundary

MDM is an enterprise security control. A legitimate Service Desk project should diagnose and document management status and use the organization's official release/unenrollment process. This project deliberately excludes bypass or unauthorized lock-removal techniques.